

1 Analyse technique et fonctionnelle complète

1.1 Projet Rapprochement bancaire, Multi-Banking et Fraud Detection

Version du rapport : 1.0

Date de l'analyse : 15 septembre 2026

Mode d'analyse : lecture seule du repository

Périmètre : code source, configuration, tests, infrastructure et documentation disponibles localement

Note méthodologique : ce rapport décrit l'implémentation réellement trouvée dans le repository. Les éléments absents, externes ou non vérifiables sont explicitement signalés.

1.2 Sommaire

1. [Résumé exécutif](#)
2. [Architecture du repository](#)
3. [Technologies et stack](#)
4. [Architecture globale et points d'entrée](#)
5. [Analyse des modules et du code](#)
6. [Rapprochement bancaire et matching](#)
7. [Flux de données de bout en bout](#)
8. [API et frontend](#)
9. [Modèle de données et persistance](#)
10. [Authentification et autorisation](#)

11. [Import, export et reporting](#)
 12. [Tests, qualité et performances](#)
 13. [Audit de sécurité](#)
 14. [Fonctionnalités et cartographie fonctionnalité-code](#)
 15. [Scénarios de soutenance](#)
 16. [Limites et plan de priorités](#)
 17. [Questions et réponses de soutenance](#)
 18. [Conclusion](#)
-

1.3 1. Résumé exécutif

Le repository contient une plateforme composée de deux services principaux :

- **Multi-Banking** : import, parsing, normalisation et validation de fichiers bancaires ;
- **Fraud Detection** : analyse hybride par règles métier, modèles ML, explicabilité SHAP et graphe Neo4j.

Le frontend est une application Angular avec SSR, servie par Express. Express agit également comme gateway vers les deux APIs FastAPI.

Le flux principal est :

```
Fichier bancaire
-> Angular / Express
-> Multi-Banking
-> PivotTransaction
-> Fraud Detection
-> Supabase / Neo4j
-> Dashboard, transactions et rapports
```

Conclusion fonctionnelle majeure : le moteur de rapprochement bancaire complet n'est pas implémenté localement. L'intégration prévue appelle un service externe BankMatch via `POST /api/import`, puis `/reconciliation/sessions/{session_id}/matching/start`.

Le champ local `reconciliationStatus` ne provient pas d'un matching réel. Il est calculé heuristiquement à partir du risque de fraude et d'un seuil de montant.

1.4 2. Architecture du repository

1.4.1 2.1 Arborescence logique

rapprochement-bancaire/	
├─ fraud-detection/	
├─ backend/	API FastAPI, règles, ML, Neo4j
├─ frontend/	Angular SSR et gateway Express
└─ e2e/	Tests end-to-end
└─ monitoring/	Prometheus
└─ docker-compose.yml	Stack Docker
├─ multi-banking/	Service d'ingestion FastAPI
├─ parsers/	CSV, CAMT.053, MT940, pain.001
├─ tests/	Tests des parseurs et intégration
├─ scripts/	Scripts utilitaires
└─ bankmatch_client.py	Client BankMatch externe
├─ docs/	Documentation technique et intégration
├─ documents/	Guides, checklist et rapports
├─ jeux-de-donnees-import/	Jeux de données et guides d'import
├─ assets/, images/	Captures et ressources
└─ scripts et tests racine	Conversion, données et vérifications

1.4.2 2.2 Dossiers importants

Dossier	Rôle réel
multi-banking	Ingestion et normalisation bancaire
fraud-detection/backend	API et logique de détection de fraude
fraud-detection/frontend	Interface Angular, SSR et gateway
fraud-detection/e2e	Tests d'intégration de la stack
fraud-detection/monitoring	Configuration Prometheus
docs , documents	Documentation et préparation d'intégration
jeux-de-donnees-import	Données et consignes d'import

1.4.3 2.3 Absences importantes

- Aucun backend BankMatch central trouvé.
- Aucun schéma SQL Supabase versionné trouvé.
- Aucune migration Supabase complète trouvée.
- Aucun moteur local de matching entre relevés et écritures trouvé.
- Aucun repository applicatif classique séparé dans le backend : une grande partie des responsabilités est concentrée dans `fraud-detection/backend/main.py`.

1.5 3. Technologies et stack

1.5.1 3.1 Backend Multi-Banking

Confirmé dans `multi-banking/requirements.txt` :

- Python 3.11 ;
- FastAPI et Uvicorn ;
- Pydantic ;
- HTTPX ;
- PyJWT ;
- `python-multipart` ;
- `lxml` ;
- Pytest et `pytest-asyncio` ;
- Prometheus FastAPI Instrumentator.

1.5.2 3.2 Backend Fraud Detection

Confirmé dans `fraud-detection/backend/requirements.txt` :

- FastAPI et Uvicorn ;
- Supabase ;
- Neo4j ;
- NumPy et Pandas ;
- scikit-learn ;
- XGBoost ;
- SHAP ;
- Joblib ;
- PyJWT ;
- PyOTP ;
- SlowAPI ;
- Prometheus.

1.5.3 3.3 Frontend

Confirmé dans `fraud-detection/frontend/package.json` :

- Angular 21 ;
- TypeScript 5.9 ;
- Angular SSR ;
- Express 5 ;
- RxJS ;
- Axios ;
- Chart.js ;
- Vis Network ;
- jsPDF et AutoTable ;

- Multer ;
- Tailwind CSS ;
- Vitest.

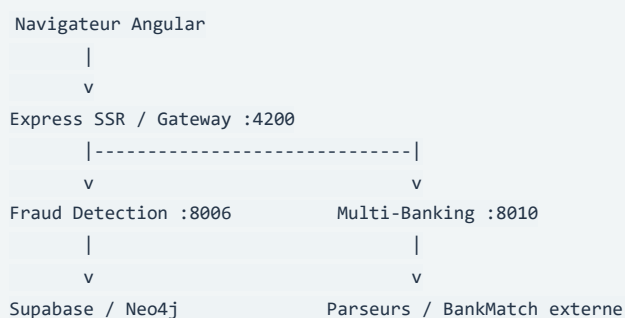
1.5.4 3.4 Infrastructure

Confirmé dans `fraud-detection/docker-compose.yml` :

- Docker Compose ;
- Nginx ;
- Neo4j 5.26 ;
- Prometheus ;
- Grafana ;
- Supabase externe.

1.6 4. Architecture globale et points d'entrée

1.6.1 4.1 Architecture réelle



Nginx fournit également un routage vers les deux services dans `fraud-detection/nginx.conf`.

1.6.2 4.2 Points d'entrée

1.6.2.1 Multi-Banking

Fichier : `multi-banking/main.py`

Objet : `app = FastAPI(...)`

Port local : `8010`

Fonctions principales :

- `parse_content()` ;
- `build_fraud_payload()` ;
- `parse_file()` ;
- `validate_file()` ;
- `ingest_file()`.

1.6.2.2 Fraud Detection

Fichier : `fraud-detection/backend/main.py`

Objet : `app = FastAPI(...)`

`root_path = "/fraud"`

Port Docker : `8006`

Fonctions principales :

- `get_service_context()` ;
- `preprocess_transaction()` ;
- `extract_rule_evaluation()` ;
- `build_transaction_output()` ;
- `analyze_batch()` ;
- `analyze_transactions_secure()` .

1.6.2.3 Frontend

- `fraud-detection/frontend/src/main.ts` : bootstrap Angular ;
- `fraud-detection/frontend/src/main.server.ts` : bootstrap SSR ;
- `fraud-detection/frontend/src/server.ts` : Express SSR et gateway.

1.6.3 4.3 Routes Angular

Définies dans `fraud-detection/frontend/src/app/app.routes.ts` :

- `/fraud-detection` ;
- `/transactions` ;
- `/reports` ;
- `/multi-banking` ;
- `/use-cases` .

1.7 5. Analyse des modules et du code

1.7.1 5.1 Multi-Banking

`multi-banking/main.py` orchestre le service :

1. réception du multipart ;
2. authentification interservice ;
3. lecture du fichier ;
4. sélection du parseur ;
5. normalisation ;
6. validation ou transmission ;
7. appel Fraud Detection ;
8. appel BankMatch optionnel ;
9. mise à jour des statistiques mémoire.

1.7.2 5.2 Modèle pivot

`multi-banking/models.py` définit `PivotTransaction` avec :

- `tenant_id` ;
- `bank_id` ;
- `account_iban` ;
- `value_date` ;
- `label` ;
- `amount` ;
- `currency` ;
- `counterparty_iban` ;
- `reference` ;
- `source_format` ;
- `balance_before` et `balance_after`.

`compute_hash()` produit un SHA-256 basé sur le tenant, le compte, la date, le montant, le libellé et la référence.

1.7.3 5.3 Parseurs

Fichier	Logique
<code>parsers/csv_bank.py</code>	Lit les colonnes CSV et crée les transactions
<code>parsers/camt053.py</code>	Namespace XML, IBAN, solde OPBD, entrées Ntry
<code>parsers/mt940.py</code>	Tags :25:, :60F:, :61:, :86:
<code>parsers/pain001.py</code>	Ordres ISO 20022, IBAN, montant et référence

CAMT.053 et MT940 reconstruisent les soldes. pain.001 représente surtout des ordres de virement et ne fournit pas nécessairement de solde.

1.7.4 5.4 Validation

`multi-banking/validators.py` expose :

- `validate_transactions()` ;
- `filter_unique_transactions()`.

Contrôles :

- IBAN obligatoire ;
- date ISO ;
- montant non nul ;
- doublon exact ;
- doublon historique ;
- tolérance de montant de 0,02 €.

La validation historique n'est pas alimentée par l'endpoint courant `validate_file()`, qui appelle la fonction sans historique externe.

1.7.5 5.5 Fraud Detection

`fraud-detection/backend/main.py` combine :

- règles métier de `rules_engine.py` ;
- features de `features.py` ;
- modèle supervisé ;
- Isolation Forest ;
- SHAP ;
- Neo4j ;
- Supabase ;
- notifications SSE.

1.7.6 5.6 Moteur de règles

`rules_engine.py` contient notamment :

- `apply_business_rules()` ;
- `apply_batch_rules()` ;
- `check_atypical_time()` ;
- `check_abnormal_velocity()` ;
- `check_device_change()` ;
- `check_geolocation_change()` ;
- `validate_transaction_sanity()`.

Règles : seuil réglementaire, montant proche du seuil, cash-out, montant anormal, mots-clés sensibles, compte dormant, nouvel IBAN, horaire atypique, vitesse, device, géolocalisation, doublons et fractionnement.

1.7.7 5.7 Graphe Neo4j

`graph_engine.py` définit `GraphEngine` et gère :

- synchronisation des comptes et transactions ;
- réseaux de fraude ;
- cycles ;
- flux réciproques ;
- comptes mules ;
- comptes les plus signalés ;
- réseau d'un compte ;
- approximation PageRank ;
- communautés approximatives.

1.7.8 5.8 Frontend

Services principaux :

- `MultiBankingService` ;
- `FraudAlertsService` ;

- TransactionsService ;
- ReportsService ;
- NotificationsService ;
- AuthService ;
- DataRefreshService .

1.8 6. Rapprochement bancaire et matching

1.8.1 6.1 Ce qui est réellement implémenté

Le flux local est :

```
Fichier
-> parsing
-> normalisation
-> validation
-> analyse fraude
-> appel BankMatch optionnel
```

1.8.2 6.2 Matching externe

Dans multi-banking/bankmatch_client.py :

```
POST {BANKMATCH_BASE_URL}/import
    -> session_id
POST {BANKMATCH_BASE_URL}/reconciliation/sessions/{session_id}/matching/start
```

Le service BankMatch central et son algorithme ne sont pas présents dans le repository.

1.8.3 6.3 Statut local de rapprochement

Dans build_transaction_output() :

```
is_fraud = true          -> SUSPICIOUS
sinon montant > 5 000 €  -> UNMATCHED
sinon                    -> MATCHED
```

Ce statut n'est donc pas calculé par comparaison avec une écriture comptable.

1.8.4 6.4 Non trouvé dans le repository

- matching exact ;
- matching fuzzy ;
- score de similarité ;
- comparaison date/référence/libellé ;
- résolution de conflits ;
- PARTIAL_MATCH calculé ;
- validation humaine locale ;

- table locale de rapprochements.

1.9 7. Flux de données de bout en bout

1.9.1 7.1 Import bancaire

```
Utilisateur
-> MultiBankingDashboardComponent.uploadFile()
-> MultiBankingService.ingestFile()
-> Express/Multer
-> multi-banking.ingest_file()
-> parse_content()
-> PivotTransaction
-> build_fraud_payload()
-> Fraud Detection /api/analyze
-> Supabase fraud_alerts
-> réponse frontend
-> DataRefreshService.trigger()
```

1.9.2 7.2 Analyse fraude

```
TransactionInput
-> apply_batch_rules()
-> agrégats Supabase
-> historique bénéficiaires
-> apply_business_rules()
-> preprocess_transaction()
-> modèle ML / SHAP / Isolation Forest
-> synchronisation Neo4j
-> TransactionOutput
```

1.9.3 7.3 Rafraîchissement UI

Après succès d'import :

```
DataRefreshService.trigger()
-> FraudDashboardComponent
-> TransactionsListComponent
-> ReportsComponent
```

1.9.4 7.4 Erreurs

Situation	Résultat
Fichier vide	HTTP 400
Format inconnu	HTTP 400
Parser invalide	HTTP 400
Fraud Detection indisponible	retries puis HTTP 502
Supabase absent	résultat non persisté ou liste vide
Neo4j absent	graphe désactivé ou données mockées
BankMatch en erreur	résultat BankMatch en erreur, fraude potentiellement conservée

1.10 8. API et frontend

1.10.1 8.1 API Fraud Detection

Routes principales :

- `POST /api/analyze ;`
- `POST /api/analyze-demo ;`
- `GET/PUT /api/config/thresholds ;`
- `GET /api/graph/* ;`
- `GET/POST/PUT/DELETE /api/transactions ;`
- `GET /api/reports/* ;`
- `GET /api/notifications ;`
- `GET /api/notifications/stream ;`
- `POST/GET /api/2fa/* ;`
- `GET /api/user/me .`

1.10.2 8.2 API Multi-Banking

- `GET /health ;`
- `GET /banking/stats ;`
- `GET /banking/uploads ;`
- `POST /banking/api/multi-banking/parse ;`
- `POST /banking/api/multi-banking/validate ;`
- `POST /banking/api/multi-banking/ingest .`

1.10.3 8.3 Proxy

`frontend/src/server.ts` réécrit :

```
/api/banking/* -> /banking/* -> Multi-Banking :8010  
/api/*         -> /fraud/api/* -> Fraud Detection :8006
```

frontend/proxy.conf.json fournit le routage équivalent en développement Angular.

1.10.4 8.4 Écarts de contrat

- plusieurs documents mentionnent encore 8005 alors que Docker et gateway utilisent 8006 ;
- FraudAlertsService appelle /api/analyze-demo ;
- le client OpenAPI appelle /api/analyze ;
- AuthService.getUserFromAPI() semble lire une réponse utilisateur directe alors que le backend enveloppe la réponse dans data ;
- l'E2E utilise parfois une route sans préfixe /banking .

1.11 9. Modèle de données et persistance

1.11.1 9.1 Supabase

Tables déduites du code :

- fraud_alerts ;
- account_aggregates ;
- beneficiary_history ;
- bank_statement_lines ;
- accounting_entries .

Le schéma SQL exact, les contraintes, les index et les politiques RLS ne sont pas présents localement.

1.11.2 9.2 fraud_alerts

Champs utilisés :

- tenant_id ;
- transaction_id ;
- transaction_reference ;
- date ;
- amount ;
- is_fraud ;
- fraud_probability ;
- score ;
- reconciliation_status ;
- rule_category ;
- explainability ;
- description ;
- bénéficiaire.

1.11.3 9.3 Neo4j

Nœuds : `Account`, `Transaction`, `Alert`.

Relations : `SENT`, `RECEIVED_BY`, `FLAGS`.

Les requêtes utilisent généralement `tenant_id` dans les motifs Cypher.

1.11.4 9.4 États mémoire

- `upload_stats` et `recent_uploads` ;
- caches vitesse/device/géolocalisation ;
- connexions SSE ;
- secrets 2FA.

Ils sont perdus au redémarrage et non partagés entre workers.

1.11.5 9.5 Cohérence multi-bases

Aucune transaction distribuée Supabase/Neo4j n'a été trouvée. Une suppression Supabase ne supprime pas nécessairement le graphe Neo4j.

1.12 10. Authentification et autorisation

1.12.1 10.1 Frontend

`auth.interceptor.ts` ajoute un Bearer token provenant de `localStorage` ou d'une session Supabase.

1.12.2 10.2 Multi-Banking

`internal_auth.py` vérifie :

- Bearer ;
- signature HS256 ;
- `type = internal` ;
- `tenantId`.

Le mode `DISABLE_INTERNAL_AUTH=true` contourne cette vérification.

1.12.3 10.3 Fraud Detection

`get_service_context()` vérifie le JWT interne avec `FRAUD_INTERNAL_SECRET`.

`get_optional_context()` accepte l'absence ou l'invalidité d'un token et retourne un contexte de développement.

1.12.4 10.4 2FA

`TwoFactorAuthService` gère TOTP et codes de secours en mémoire. La liaison à une authentification utilisateur obligatoire n'est pas confirmée.

1.13 11. Import, export et reporting

1.13.1 11.1 Imports

Formats confirmés :

- CSV ;
- CAMT.053 ;
- MT940 ;
- pain.001.

1.13.2 11.2 Exports

- CSV via `/api/reports/csv` ;
- HTML compatible impression PDF via `/api/reports/pdf` .

L'endpoint nommé PDF ne produit pas un PDF binaire natif : il renvoie du HTML.

1.13.3 11.3 Rapports

`/api/reports` calcule :

- total transactions ;
- nombre de fraudes ;
- taux de fraude ;
- montant bloqué ;
- catégories ;
- séries temporelles.

Les données sont lues depuis `fraud_alerts` et dédoublées par référence.

1.14 12. Tests, qualité et performances

1.14.1 12.1 Tests présents

- tests parseurs Multi-Banking ;
- règles métier ;
- API backend ;
- ML et fusion des scores ;
- persistance mockée ;
- graphes mockés ;
- authentification ;
- 2FA ;
- services Angular ;
- tests E2E.

1.14.2 12.2 Lacunes

- Supabase réel non testé ;
- Neo4j réel non testé ;
- BankMatch réel non testé ;
- aucun test navigateur complet ;
- aucun test de charge ;
- pas de couverture minimale bloquante ;
- CI probablement mal placée dans les sous-dossiers ;
- incohérences de ports/routes dans E2E ;
- isolation tenant insuffisamment testée.

1.14.3 12.3 Performance

Validation des doublons : complexité potentielle :

$O(n \times m)$

où n est le nombre de transactions entrantes et m la taille de l'historique.

Autres coûts :

- fichier entier chargé en mémoire ;
- liste complète des transactions en mémoire ;
- appels Neo4j par transaction ;
- retries HTTP longs ;
- caches globaux sans TTL ;
- statistiques d'upload non persistantes.

1.15 13. Audit de sécurité

1.15.1 13.1 Critique

1. `get_optional_context()` permet des appels sans authentification.
2. `DISABLE_INTERNAL_AUTH=true` est présent dans Docker.
3. Plusieurs endpoints acceptent `tenant_id` fourni par le client.
4. Les fallbacks CRUD retirent le filtre tenant.
5. Les routes 2FA ne montrent pas d'authentification obligatoire.
6. Des secrets et mots de passe de développement sont présents dans les `.env` /Compose.
7. `INTERNAL_SERVICE_SECRET` est écrit dans les logs Multi-Banking.

1.15.2 13.2 Élevé

- tenant du formulaire non comparé au tenant JWT ;
- uploads non limités directement dans FastAPI ;
- notifications SSE globales ;

- données historiques consultées sans filtre tenant explicite dans `analyze_batch()` ;
- Neo4j, Grafana et Prometheus publiés par Docker ;
- client `service_role` Supabase possible.

1.15.3 13.3 Moyen

- CORS large avec credentials ;
- token de test activé hors production ;
- HTML d'export construit par interpolation ;
- dépendances non verrouillées ;
- images Docker `latest` ;
- rate limiting incomplet ;
- conteneurs sans utilisateur non-root visible.

1.15.4 13.4 Non confirmable localement

- politiques RLS Supabase réelles ;
- exposition Internet effective ;
- historique Git des secrets ;
- CVE précises sans scanner ;
- contrôles externes non présents.

1.16 14. Fonctionnalités et cartographie fonctionnalité-code

Fonctionnalité	Composant frontend	API	Code principal	Stockage
Import CSV	<code>MultiBankingDashboardComponent</code>	<code>/banking/api/multi-banking/ingest</code>	<code>parse_csv()</code>	<code>fraud_alerts</code> après analyse
Validation	<code>MultiBankingService</code>	<code>/validate</code>	<code>validate_transactions()</code>	aucun direct
Analyse fraude	<code>FraudAlertsService</code>	<code>/api/analyze</code>	<code>analyze_batch()</code>	<code>fraud_alerts</code>
Règles	Dashboard fraude	<code>/api/analyze</code>	<code>apply_business_rules()</code>	<code>thresholds.json</code>
ML	Dashboard fraude	<code>/api/analyze</code>	<code>preprocess_transaction()</code>	fichiers <code>.pkl</code>
Graphe	<code>GraphService</code>	<code>/api/graph/*</code>	<code>GraphEngine</code>	Neo4j
Transactions	<code>TransactionsService</code>	<code>/api/transactions</code>	CRUD FastAPI	<code>fraud_alerts</code>
Rapports	<code>ReportsService</code>	<code>/api/reports/*</code>	fonctions reports	<code>fraud_alerts</code>
Notifications	<code>NotificationsService</code>	<code>/api/notifications*</code>	<code>SSEManager</code>	mémoire
Matching	dashboard Multi-Banking	BankMatch externe	non présent localement	BankMatch

1.17 15. Scénarios de soutenance

1.17.1 Scénario A — Import réussi

```
Utilisateur  
-> composant Angular  
-> service Angular  
-> gateway Express  
-> parseur Multi-Banking  
-> PivotTransaction  
-> Fraud Detection  
-> Supabase  
-> réponse  
-> rafraîchissement global
```

1.17.2 Scénario B — Fraude détectée

Une transaction dépassant le seuil réglementaire ou contenant un mot-clé sensible déclenche une règle. Le score est fusionné avec le ML. Le résultat peut être persisté dans Supabase, synchronisé dans Neo4j et diffusé par SSE.

1.17.3 Scénario C — Aucun match bancaire

Le repository local ne réalise pas la recherche de correspondance. Le statut `UNMATCHED` peut seulement provenir du seuil de montant local ou du résultat BankMatch externe non visible ici.

1.17.4 Scénario D — Plusieurs candidats

Non trouvé dans le code local. La résolution doit être réalisée par BankMatch.

1.17.5 Scénario E — Erreur d'import

Le parseur lève une erreur, Multi-Banking renvoie `400`, le frontend traduit l'erreur et affiche un toast.

1.18 16. Limites et plan de priorités

1.18.1 Priorité 1 — Sécurité

- imposer l'authentification en production ;
- faire tourner les secrets ;
- supprimer les fallbacks sans tenant ;
- protéger la 2FA ;
- isoler SSE et historiques par tenant ;
- désactiver les modes développement.

1.18.2 Priorité 2 — Contrats

- unifier `8005` / `8006` ;

- aligner OpenAPI, routes et frontend ;
- corriger les E2E ;
- finaliser le contrat BankMatch ;
- versionner le schéma Supabase.

1.18.3 Priorité 3 — Robustesse

- limite d'upload côté FastAPI ;
- streaming des gros fichiers ;
- persistance des uploads ;
- stratégie de cohérence Supabase/Neo4j ;
- suppression des mocks en production.

1.18.4 Priorité 4 — Performance

- index tenant ;
- déduplication structurée ;
- écritures Neo4j par lot ;
- Redis pour les caches ;
- tests de charge.

1.18.5 Priorité 5 — Maintenabilité

- découper `main.py` en routes/services/repositories ;
- mutualiser les contrats DTO ;
- ajouter CI frontend et Multi-Banking ;
- ajouter des tests d'intégration réels.

1.19 17. Questions et réponses de soutenance

1.19.1 Pourquoi deux services backend ?

Pour séparer l'ingestion des formats bancaires de l'analyse fraude. Multi-Banking produit un modèle pivot ; Fraud Detection consomme ce modèle standardisé.

1.19.2 Le projet réalise-t-il le rapprochement ?

Il prépare et transmet les transactions à BankMatch. Le moteur de matching complet n'est pas local. `reconciliationStatus` est une classification heuristique locale.

1.19.3 Pourquoi un modèle pivot ?

Pour découpler les formats CSV, CAMT.053, MT940 et pain.001 des consommateurs aval.

1.19.4 Comment le score est-il calculé ?

Par fusion de règles métier, modèle supervisé, Isolation Forest et explicabilité SHAP.

1.19.5 À quoi sert Neo4j ?

À représenter et analyser les relations entre comptes et transactions : cycles, flux réciproques, réseaux et comptes mules.

1.19.6 Où sont stockées les données ?

Supabase conserve les alertes et historiques. Neo4j conserve le graphe. Les seuils sont dans `thresholds.json`. Plusieurs états opérationnels sont en mémoire.

1.19.7 Quelles sont les principales limites ?

Matching externe, schéma Supabase absent, sécurité de développement permissive, états mémoire, tests externes mockés, incohérences de ports et absence de benchmark de charge.

1.19.8 Quelle amélioration prioritaire ?

Rendre l'authentification et l'isolation tenant obligatoires avant toute exposition en production.

1.20 18. Conclusion

Le repository implémente une chaîne complète d'ingestion bancaire multi-format et d'analyse de fraude hybride, avec interface Angular, APIs FastAPI, persistance Supabase et analyse relationnelle Neo4j.

La partie la plus aboutie localement est l'ingestion et l'analyse fraude. Le rapprochement bancaire réel reste une responsabilité externe de BankMatch. Cette distinction doit être présentée clairement lors d'une soutenance afin de ne pas attribuer au code local une fonctionnalité qu'il ne contient pas.

Avant une mise en production, les travaux prioritaires sont la sécurité multi-tenant, la rotation des secrets, la cohérence des contrats, la persistance durable des états, les tests d'infrastructure et la mesure des performances.